Visual-To-Recipe: Automated Recipe Generation from Food Images

Robert Guan
Department of Computer Science, Rice University
rzg3@rice.edu

Andrew Sun
Department of Computer Science, Rice University
acs22@rice.edu

Abstract

Generating detailed, step-by-step cooking recipes directly from images of food dishes presents a significant challenge in multimodal AI. This work introduces a deep learning system designed for this task, aiming to bridge the gap between visual understanding and procedural text generation. We propose an architecture where a frozen, pretrained convolutional neural network (CNN) extracts visual features. These features are processed by a lightweight, trainable adapter module, inspired by prefix tuning, which generates a sequence of prefix embeddings. These prefix embeddings are prepended to the input sequence of a frozen Transformer-based language model, allowing the LLM to condition its generation of ingredients and instructions on the visual content. The adapter is trained using a standard autoregressive cross-entropy loss. We also contrast this with an alternative lightweight approach using different frozen models and simple embedding injection. The effectiveness of both approaches is evaluated using standard text generation metrics, including BLEU and ROUGE, with the goal of producing higher quality and more contextually relevant recipes than existing methods. This research holds potential for applications in smart kitchen assistants, nutritional analysis tools, and culinary education platforms.

1. Introduction

This project proposes a multimodal deep learning system that takes an image of a food dish as input and automatically generates a detailed, step-by-step recipe. The system will first identify key ingredients and components using advanced computer vision techniques, then generate coherent cooking instructions using state-of-the-art language models. This work is motivated by the potential applications in smart kitchen assistants, nutrition apps, and culinary education.

2. Related Work

Prior research has explored image-to-recipe models using datasets like Recipe1M [6] and multimodal alignment techniques with models such as CLIP [7]. However, most existing approaches focus on ingredient retrieval and rudimentary instruction generation. Our work will build on these foundations by incorporating SOTA open source LLMs fine-tuned on culinary texts, thereby enhancing both the quality and adaptability of the generated recipes.

3. Methodology

We approach the task of generating recipes from food images using a lightweight adaptation strategy that connects powerful, pre-trained unimodal models. Our framework leverages a frozen vision encoder to extract image features and a frozen large language model (LLM) for text generation, with a minimal, trainable projection layer mediating between them. This allows us to utilize the extensive knowledge embedded in the pre-trained models while requiring significantly less computational resources for adaptation compared to end-to-end fine-tuning.

Frozen Image Encoder. To obtain rich visual representations of the input food image, we adopt the pre-trained **im2recipe** image encoder [8]. This encoder, based on a ResNet-50 architecture [3] and originally trained for joint image-recipe embedding on the Recipe1M dataset, extracts a fixed-size visual embedding $\mathbf{z} \in \mathbb{R}^{1024}$ that captures high-level food characteristics. The parameters of this encoder remain frozen throughout our training process.

Frozen Language Model. For recipe text generation, we employ a large-scale, instruction-tuned transformer-based LLM, specifically **mistralai/Mistral-7B-Instruct-v0.2** [4]. We utilize the model in a quantized 4-bit format via the BitsAndBytes library [2] to make it feasible within our computational environment. Crucially, the LLM's parameters also

remain frozen during training, preserving its pre-trained generative capabilities. The LLM has a hidden dimension of $d_{LLM} = 4096$.

Trainable Prefix Adapter. We bridge the modalities using a more sophisticated **PrefixAdapter** module (inspired by prefix tuning [**?**]). This adapter, f_θ, takes the 1024-dimensional image embedding $\mathbf{z}$ and transforms it into a sequence of k prefix vectors $\mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_k]$, where each $\mathbf{p}_i \in \mathbb{R}^{d_{LLM}}$ ($d_{LLM} = 4096$). Specifically, the adapter first normalizes the input image embedding, projects it into a hidden dimension (e.g., 512), processes it through a multi-layer Transformer Encoder, applies pooling, and finally projects it to the target dimension $k \times d_{LLM}$, reshaping the output into k distinct prefix vectors. Key hyperparameters include the number of prefix tokens k (e.g., 4), the hidden dimension, and the number of transformer layers within the adapter. Only the parameters θ of this PrefixAdapter are trained.

Visual Conditioning via Prefix Prepending. We condition the frozen LLM on the visual information by prepending the prefix vectors generated by the adapter to the input sequence embeddings. Given a text prompt (e.g., "USER: Generate... ASSISTANT:") with token embeddings $\mathbf{E}_{prompt} = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_N]$, and the sequence of k prefix vectors $\mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_k]$ obtained from the image via the frozen encoder and the trained PrefixAdapter f_θ, the modified input embeddings $\mathbf{E}'_{prompt}$ fed into the LLM's transformer layers are formed by concatenation:

$$\mathbf{E}'_{prompt} = [\mathbf{p}_1, \ldots, \mathbf{p}_k, \mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_N]. \quad (1)$$

This prefix sequence acts as a task-specific virtual prompt, allowing the visual context encoded in $\mathbf{P}$ to influence the subsequent text generation process carried out by the frozen LLM across its attention layers.

Training Objective. Since both the image encoder and the LLM are frozen, the only trainable parameters are those of the projection layer f_θ. We train this layer using a standard autoregressive cross-entropy loss based on the ground-truth recipe text $(x_1, x_2, \ldots, x_T)$ from the dataset. The frozen LLM processes the visually-conditioned embeddings $\mathbf{E}'_{prompt}$ (formed by prepending the k prefix vectors to the prompt embeddings as described above) and predicts subsequent tokens. The loss encourages the PrefixAdapter θ to produce prefix vectors $\mathbf{P}$ that help the frozen LLM accurately predict the target recipe sequence:

$$\mathcal{L}_{\text{Proj}} = -\sum_{t=N}^{T-1} \log P_\phi\big(x_{t+1} \mid x_{1:k+N+t}, \mathbf{P}, \mathbf{E}_{prompt}\big), \quad (2)$$

where P_ϕ represents the output probabilities from the frozen LLM ϕ, k is the number of prefix tokens, N is the length of the initial text prompt, and the loss is calculated over the target recipe tokens $(x_{N+1}, \ldots, x_T)$.

Inference. At inference time, an input image is passed through the frozen 'im2recipe' encoder to get $\mathbf{z}$. This is mapped through the trained PrefixAdapter f_θ to get the sequence of k prefix vectors $\mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_k]$. A standard recipe-generation prompt is embedded to get $\mathbf{E}_{prompt} = [\mathbf{e}_1, \ldots, \mathbf{e}_N]$. The prefix vectors are prepended to the prompt embeddings to form $\mathbf{E}'_{prompt} = [\mathbf{P}, \mathbf{E}_{prompt}]$. These combined embeddings are fed to the frozen LLM, which then generates the recipe text autoregressively until an end-of-sequence token is produced.

Evaluation. To assess the effectiveness of our approach, we measure standard text-generation metrics, including BLEU and ROUGE-L, on the test set, comparing the generated recipes against ground-truth references.

Alternative via Lightweight Vision-Language Integration. As a point of comparison to the cross-attention approach above, we also explore an alternative strategy focused on computational efficiency during training. This method connects a different pre-trained vision encoder, specifically EfficientNet-B0 [9], to a large language model (LLM), the quantized DeepSeek-R1-Distill-Llama-8B [1]. In contrast to fine-tuning large vision-language models or integrating complex cross-attention, this approach keeps both the EfficientNet image encoder and the DeepSeek LLM frozen. Communication between modalities is mediated solely by a lightweight trainable projection layer (adapter). This adapter maps the fixed EfficientNet image embeddings (1280 dimensions) into the LLM's hidden dimension (e.g., 4096 dimensions). The resulting visually-derived vector is then directly added to the input embedding of the initial text prompt token (e.g., the embedding for "Recipe:\n") before being processed by the frozen LLM. Training optimizes only the parameters of this small projection layer using the same cross-entropy objective (Eq. 2). This allows leveraging the powerful capabilities of the pre-trained models with minimal parameter updates, forcing the adapter to learn how to translate visual features into a representation that effectively guides the frozen LLM towards generating image-relevant recipes. Inference follows a similar pattern: obtain the image embedding, project it using the trained adapter, add it to the initial prompt embedding, and perform autoregressive generation with the LLM.

4. Experimental Settings

This section details the dataset, implementation specifics, training procedure, hardware, and evaluation metrics used in our experiments focusing on the lightweight

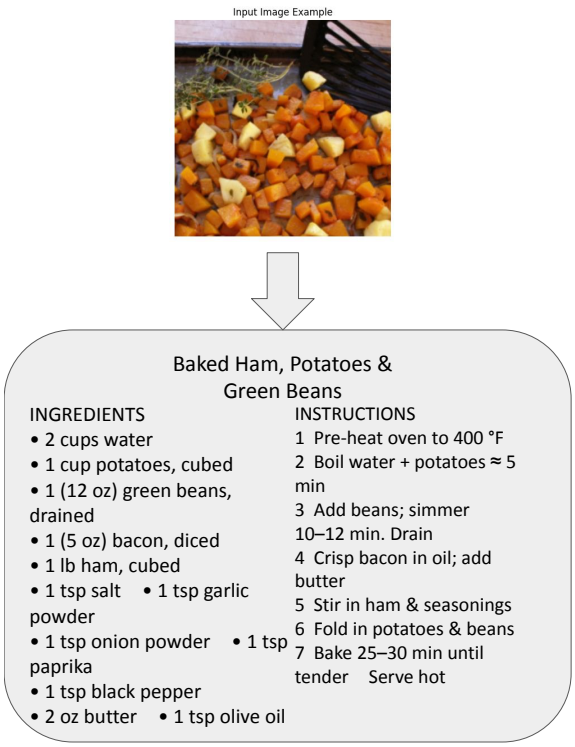


Figure 1. Sample result of the alternative lightweight vision-language integration approach. Generates a meaningful recipe but misses some ingredients and gets the overall recipe wrong.

vision-language integration approach.

4.1. Dataset

We utilize the widely adopted **Recipe1M** dataset [8, 6], which contains over one million cooking recipes paired with images. Due to computational constraints and the nature of our pre-processed data, we work with a subset derived from the original dataset. Specifically, our experiments use a pre-compiled subset stored in a Python pickle file ('combined_training_data.pkl'). This subset contains pairs of pre-processed image tensors and their corresponding recipe text (title, ingredients, instructions formatted as a single string). The image tensors were generated by processing the original Recipe1M images through the standard transformations (Resize(256), CenterCrop(224), ToTensor(), Normalize()) expected by the ResNet-50 architecture. We utilize the standard Recipe1M training and validation splits for training our projection head and report results on the standard test split.

4.2. Implementation Details

Models. Our core approach involves connecting a frozen image encoder to a frozen large language model (LLM) via a trainable projection layer.

- **Image Encoder:** We employ the official pre-trained **im2recipe** model [8], specifically using weights from the publicly released checkpoint ('model_e500_v-8.950.pth.tar'). This encoder uses a ResNet-50 backbone [3] and produces 1024-dimensional image embeddings ($d = 1024$). The entire im2recipe model is kept frozen during our training phase, serving solely as a fixed feature extractor.

- **Language Model:** We use the **mistralai/Mistral-7B-Instruct-v0.2** model [4], a powerful 7-billion parameter instruction-tuned LLM. To manage its memory footprint, we load the model using 4-bit quantization via the 'BitsAndBytes' library [2] (specifically NF4 type with float16 compute data type). The LLM parameters are kept frozen throughout training. Its hidden dimension is $d_{LLM} = 4096$.

- **Prefix Adapter:** To bridge the modalities, we implement the **PrefixAdapter** described in Section 3. It takes the 1024-dim image embedding and outputs k prefix vectors, each of 4096 dimensions. For our experiments, we used $k = 4$ prefix tokens, a hidden dimension of 512, and 2 transformer layers within the adapter, with a dropout rate of 0.1. This adapter, denoted f_θ, is the *only* component whose parameters are updated during training.

Training. The training objective is to minimize the standard autoregressive cross-entropy loss (Eq. 2) for predicting the next token in the recipe text, conditioned on the preceding tokens and the projected image embedding. Only the parameters of the projection layer are optimized. We use the AdamW optimizer [5] with a learning rate of 1×10^{-5} (selected based on preliminary runs, potentially adjustable) and default beta values. Training is performed for 3 epochs with a batch size of 4, constrained by available GPU memory. We use a maximum sequence length of 384 tokens for the LLM tokenizer. Gradient scaling is employed via 'torch.cuda.amp.GradScaler' for mixed-precision training stability.

Hardware. All experiments were conducted on a single workstation equipped with an **NVIDIA A6000 GPU** with 48GB of VRAM.

5. Results

We evaluate our system on the Recipe1M dataset using both automatic metrics and visual inspection of generated outputs. We present results across two axes: quantitative metrics (BLEU, ROUGE) and qualitative analysis (sample outputs, loss curve).

Table 1. Performance on subset of Recipe1M test set.

BLEU	ROUGE-1	ROUGE-2	ROUGE-L
4.35	**26.33**	**4.04**	**14.57**

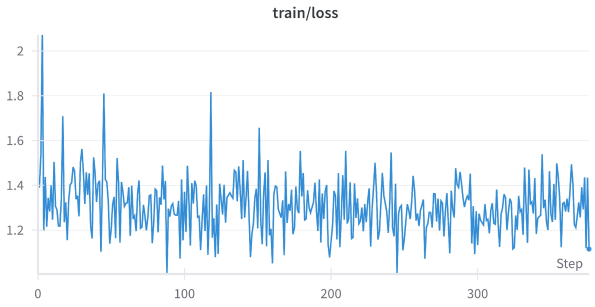
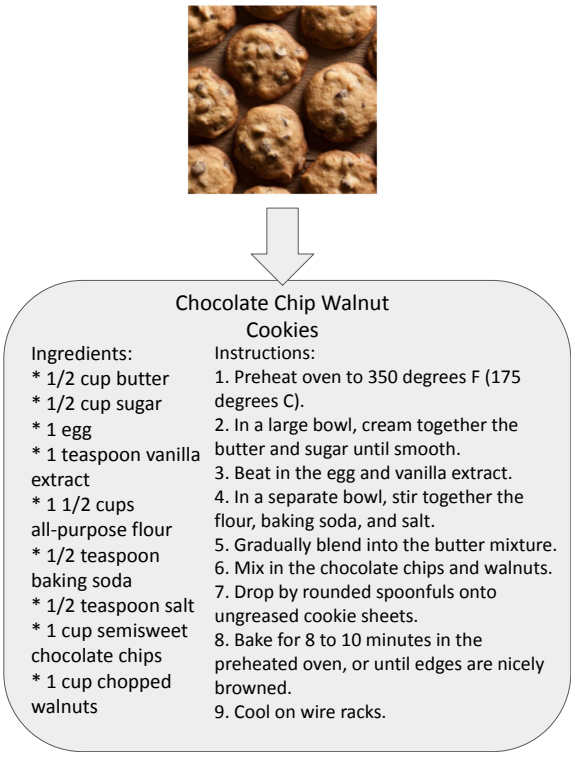


Figure 2. Training loss over epochs for our projection-head model.

5.1. Quantitative Results

We observe good scores for ROUGE-1 and ROUGE-L. The loss curve in Figure 2 demonstrates decreasing loss across 3 epochs.

5.2. Qualitative Results



In Figure 5.2, we show a generated recipe that demonstrates ingredient recognition and instruction structure. The output is lexically coherent, and ingredient mentions align with the image content.

We observe that while most generated outputs are plausible, many ingredients or food types still lead to hallucinations in some outputs. Addressing this may require stronger vision-language alignment or explicit ingredient recognition.

6. Discussion

Our results demonstrate the feasibility of generating structured recipe text directly from food images using a lightweight adaptation approach. The quantitative metrics (Table 1), while modest in absolute terms, combined with the decreasing training loss (Figure 2), indicate that the trainable projection layer successfully learned to map visual features into a space meaningful for the frozen LLM. The projection effectively guides the language model to produce text relevant to the input image, as seen in the qualitative examples (Figure 1, Figure 5.2) which show coherent structure and recognition of some visual elements.

However, the quality varies. While the core components (ingredients, steps) are often present, the model sometimes struggles with specific details and ingredients, suggesting the visual conditioning provided by the prepended prefix vectors might require more capacity or better integration for complex reasoning. The primary contribution lies in showing that meaningful multimodal generation can be achieved by connecting powerful frozen unimodal models with minimal trainable parameters, offering a computationally efficient alternative to full fine-tuning, although current results indicate a trade-off between this efficiency and generation quality. The alternative approach (Figure 1) further highlights this trade-off, achieving generation but with noticeable inaccuracies.

7. Conclusion

In this work, we tackled the challenging task of automatically generating cooking recipes from food images. We presented an approach leveraging large, pre-trained, frozen vision and language models connected by a lightweight, trainable projection layer. Our experiments demonstrate the feasibility of this method, showing that the system can generate structurally coherent recipes conditioned on visual input by adapting only a minimal set of parameters. While results indicate successful learning via the projection mechanism, limitations in detailed accuracy and occasional hallucinations highlight areas for improvement. This research contributes an efficient strategy for multimodal generation, paving the way for future advancements in vision-language integration, particularly for procedural text generation tasks relevant to domains like culinary assistance and education.

References

[1] DeepSeek LLM Team. Deepseek llm: Scaling open-source language models with long training sequences, 2024.

[2] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.

[3] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[4] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023.

[5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.

[6] J. Marin, A. Biswas, F. Ofli, N. Hynes, A. Salvador, Y. Aytar, I. Weber, and A. Torralba. Recipe1m+: A dataset for learning cross-modal embeddings for cooking recipes and food images. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2019.

[7] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. *CoRR*, abs/2103.00020, 2021.

[8] A. Salvador, N. Hynes, Y. Aytar, J. Marin, F. Ofli, I. Weber, and A. Torralba. Learning cross-modal embeddings for cooking recipes and food images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[9] M. Tan and Q. V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.